

SERVICIO NACIONAL DE APRENDIZAJE – SENA
CENTRO DE TECNOLOGÍA DE LA MANUFACTURA AVANZADA – CTMA
REGIONAL ANTIOQUIA

PROGRAMA: Tecnólogo en Análisis y Desarrollo de Software (ADSO)

FICHA: 3229209

Actividad General 03 Sept

Módulo — DevOps y Contenedores

Presentado por:

Manuela Cordoba Robledo

Presentado a:

Richard Alexander Betancur Sierra

Medellín, Antioquia

Septiembre de 2

Taller DevOps y Contenedores — Módulo 2: Gestión de contenedores

1. Modos de ejecución de un contenedor

Comandos vistos (teoría del profe):

docker run -it --name consola ubuntu:24.04 bash
docker run -d --name web -p 8080:80 nginx
docker logs -f web
docker exec -it web bash
docker exec web nginx -v

2. Práctica — Resolución de conflicto de puertos

Imagen 1 — Estado inicial de contenedores en Docker Desktop

```
PS C:\WINDOWS\system32> docker ps
```

CONTAINER ID	IMAGE	COMMAND
CREATED	STATUS	PORTS
	NAMES	
95c6b2224891	ejercicio-app	"uvicorn main:app
---"	9 days ago Up About a minute (healthy)	0.0.0.0:8001->5000/t
cp, [::]:8001->5000/tcp	ejercicio3_app	
618a3ca8f9a7	postgres:15-alpine	"docker-entrypoin
t.s..."	9 days ago Up About a minute (healthy)	0.0.0.0:5434->5432/t
cp, [::]:5434->5432/tcp	ejercicio3_postgres	
cf67fad6a3d8	redis:7-alpine	"docker-entrypoin
t.s..."	9 days ago Up About a minute (healthy)	0.0.0.0:6380->6379/t
cp, [::]:6380->6379/tcp	ejercicio3_redis	
fed075bb114e	ejemplo-app	"python app.py"
13 days ago Up About a minute		0.0.0.0:5000->5000/t
cp, [::]:5000->5000/tcp	app_flask	
ed2685e3a93f	rediscommander/redis-commander:latest	"/usr/bin/dumb-in
it ..."	13 days ago Up About a minute (healthy)	0.0.0.0:8081->8081/t
cp, [::]:8081->8081/tcp	redis_ui	
2bbf317050a1	redis:7-alpine	"docker-entrypoin
t.s..."	13 days ago Up About a minute	0.0.0.0:6379->6379/t
cp, [::]:6379->6379/tcp	mi_redis	
8e34d2ca2c25	dpage/pgadmin4:latest	"/entrypoint.sh"
13 days ago Up About a minute		0.0.0.0:5050->80/tcp
, [::]:5050->80/tcp	ejercicio_pgadmin	
b99e6458bdaa	ejercicio-api	"docker-entrypoin
t.s..."	13 days ago Up About a minute	0.0.0.0:3001->3000/t
cp, [::]:3001->3000/tcp	ejercicio_api	
f593e1dfff5d4	ejemplo-api	"docker-entrypoin
t.s..."	13 days ago Up About a minute	0.0.0.0:3000->3000/t
cp, [::]:3000->3000/tcp	api_node	
8f28a9d4139e	postgres:15-alpine	"docker-entrypoin
t.s..."	13 days ago Up About a minute (healthy)	0.0.0.0:5432->5432/t

Imagen 2 — Error al crear el contenedor por puerto ocupado

```
PS C:\WINDOWS\system32> docker run -d --name practical -p 8081:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
f340c1b7c1d6: Download complete
6310eb16bf42: Downloading [=====] 24.12MB/29.79MB
f340c1b7c1d6: Pull complete
2: Pull complete
plete
02fc02c4ab8d: Pull complete
c12f394dea35: Pull complete
07db7bf2649b: Pull complete
76f27c02d218: Download complete
25202a7045eb: Download complete
Digest: sha256:05b8cb60c354a44ab824ea6e7dc69b46d50762cdbe728a347a5b656e6fb3d7c4
Status: Downloaded newer image for nginx:latest
6f3138568c9e4a9d223ceb74d5945ecfa624d984f3eda2e513f747fed2b8f884

What's next:
  Debug this container error with Gordon → docker ai "help me fix this
  container error"
docker: Error response from daemon: failed to set up container networkin
g: driver failed programming external connectivity on endpoint practical
```

-Intenté crear practica1 con el puerto 8081, pero Docker mostró el error porque ese puerto ya estaba en uso.

Imagen 3 — Deteniendo el contenedor en conflicto y arrancando el mío

```
PS C:\WINDOWS\system32> docker stop php-container
php-container
PS C:\WINDOWS\system32> docker start practical
practical
```

- Identifiqué que el contenedor php-container estaba usando el puerto en conflicto, lo detuve con `docker stop php-container` y luego intenté arrancar mi contenedor practica1 con `docker start`.

Imagen 4 — Verificando que el puerto no quedó publicado

```
PS C:\WINDOWS\system32> docker ps -a --filter "name=practical"
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
6f3138568c9e   nginx    "/docker-entrypoint..." 6 minutes ago  Up Abo
ut a minute    80/tcp   practical
```

- Al verificar con `docker ps -a --filter "name=practica1"`, noté que el contenedor no tenía ningún puerto publicado hacia mi máquina (columna PORTS vacía) — porque se había creado sin éxito en el mapeo desde el intento fallido inicial, así que no bastaba con solo arrancarlo.

Imagen 5 — Recreando el contenedor con un puerto libre

```
PS C:\WINDOWS\system32> docker rm -f practical
practical
PS C:\WINDOWS\system32> docker run -d --name practical -p 8082:80 nginx
c0fe309251de9c1e3f066f907e6b91f182680dd6e7487b7923caf7f0f810f64e
```

- Eliminé el contenedor y lo volví a crear usando el puerto 8082, que estaba libre.

Imagen 6 — Confirmando el mapeo correcto

```
PS C:\WINDOWS\system32> docker ps --filter "name=practical"
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
PORTS
c0fe309251de   nginx     "/docker-entrypoint...." 19 seconds ago Up 17
seconds       0.0.0.0:8082->80/tcp, [::]:8082->80/tcp   practical
```

- Verifiqué que ahora sí el contenedor tenía el puerto correctamente publicado.

Imagen 7 — Resultado final en el navegador

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

-Al entrar a <http://localhost:8082> se mostró la página de bienvenida de nginx, confirmando que el contenedor quedó funcionando correctamente.

Conclusión / lección aprendida:

Antes de publicar un puerto con -p, es importante revisar con `docker ps` cuáles puertos ya están en uso, para evitar conflictos como este.

Imagen 8 — Verificación de variables de entorno en Postgres

```
PS C:\WINDOWS\system32> docker run -d --name db -e POSTGRES_USER=admin -
e POSTGRES_PASSWORD=secreto -e POSTGRES_DB=tienda postgres:16
2a85cfc4246faeda681fa87f27479848d0d9a49c275eea89f405e0ad37c60d9d
PS C:\WINDOWS\system32> docker run -d --name db -e POSTGRES_USER=admin -
e POSTGRES_PASSWORD=secreto -e POSTGRES_DB=tienda postgres:16
2a85cfc4246faeda681fa87f27479848d0d9a49c275eea89f405e0ad37c60d9d
PS C:\WINDOWS\system32> docker exec -it db psql -U admin -d tienda -c "\
l"

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any con
tainer or image → docker debug db
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\WINDOWS\system32> docker exec -it db psql -U admin -d tienda -c "\
l" --pset pager=off

                                List of databases
   Name      | Owner | Encoding | Locale Provider | Collate  | Ctype
   | ICU Locale | ICU Rules | Access privileges |          |
-----+-----+-----+-----+-----+-----+-----
 postgres    | admin | UTF8      | libc              | en_US.utf8 | en_US.utf
8 |          |          |                   |          |
 template0   | admin | UTF8      | libc              | en_US.utf8 | en_US.utf
8 |          |          | =c/admin          |          |
 |           |      |          | admin=CTc/admin  |          |
 template1   | admin | UTF8      | libc              | en_US.utf8 | en_US.utf
8 |          |          | =c/admin          |          |
 |           |      |          | admin=CTc/admin  |          |
 tienda      | admin | UTF8      | libc              | en_US.utf8 | en_US.utf
8 |          |          |                   |          |
(4 rows)
```

```
What's next:
    Try Docker Debug for seamless, persistent debugging tools in any con
tainer or image → docker debug db
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\WINDOWS\system32> |
```

-Ejecuté `docker exec -it db psql -U admin -d tienda -c "\l" --pset pager=off` para listar las bases de datos dentro del contenedor. El resultado confirma

que las variables de entorno (POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB) funcionaron correctamente: se creó el usuario admin como dueño de la base tienda.

Imagen 9 — Publicación de puerto de Postgres.

```
PS C:\WINDOWS\system32> docker rm -f db
db
PS C:\WINDOWS\system32> docker run -d --name db -p 5432:5432 -e POSTGRES
_PASSWORD=x postgres:16
bfd2098bdc9c8831a9ab4965419da2e409a3732e0fadad82a361538265eec64d

What's next:
  Debug this container error with Gordon → docker ai "help me fix this
  container error"
docker: Error response from daemon: failed to set up container networkin
g: driver failed programming external connectivity on endpoint db (69820
81f7f9df2b1e41e4c2c3e0cc32d023461217b0ede7b7a28a5ab2ad69300): Bind for 0
.0.0.0:5432 failed: port is already allocated

Run 'docker run --help' for more information

PS C:\WINDOWS\system32> docker rm -f db
db
PS C:\WINDOWS\system32> docker run -d --name db -p 5433:5432 -e POSTGRES
_PASSWORD=x postgres:16
e5cceed70003028c0beb0856fee44e343831e3a94c0ee2dbe29cdda67090841e
PS C:\WINDOWS\system32> docker ps --filter "name=db"
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS               NAMES
e5cceed70003   postgres:16    "docker-entrypoint.s..." 26 seconds ago Up 26 seconds    0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp   db
PS C:\WINDOWS\system32>
```

-Al intentar publicar Postgres en el puerto estándar 5432, obtuve un error porque ya estaba ocupado por otro contenedor. Recreé el contenedor db usando -p 5433:5432, dejando el puerto interno del contenedor en 5432 (el que Postgres usa por defecto) y cambiando el puerto externo a 5433. Verifiqué con `docker ps --filter "name=db"` que el mapeo quedó correcto. Esta es tu **Imagen 10 — Prueba de persistencia con volumen con nombre**.

Imagen 10 — Prueba de persistencia con volumen con nombre.

```
What's next:
  Try Docker Debug for seamless, persistent debugging tools in any con
tainer or image → docker debug db
  Learn more at https://docs.docker.com/go/debug-cli/
```

```
PS C:\WINDOWS\system32> docker run -d --name db -v datos_db:/var/lib/postgresql/data -p 5433:5432 -e POSTGRES_PASSWORD=x postgres:16
7aec2b5f9a9f21ea97cd10dc2ba9b9cddb52716a1d30bc005226c7555d1cbb15
```

```
What's next:
  Try Docker Debug for seamless, persistent debugging tools in any con
tainer or image → docker debug db
  Learn more at https://docs.docker.com/go/debug-cli/
```

Imagen 11 — Bind mount funcionando.


```

PS C:\WINDOWS\system32> cd $HOME
PS C:\Users\roble> mkdir sitio

Directorio: C:\Users\roble

Mode                LastWriteTime         Length Name
----                -
d-----          3/09/2026   2:10 p. m.             sitio

PS C:\Users\roble> Set-Content -Path sitio/index.html -Value "<h1>Hola desde bind mount</h1>"
PS C:\Users\roble> docker run -d --name web2 -v ${PWD}/sitio:/usr/share/nginx/html -p 8083:80 nginx
9c6affaf2039fac2c88f9fd4871c9e36c306a19b48c784ea047f552724702b15
PS C:\Users\roble>

```

Hola desde bind mount

-Creé una carpeta local (sitio) con un archivo index.html de prueba, y la conecté al contenedor web2 mediante un bind mount (-v \${PWD}/sitio:/usr/share/nginx/html). Al entrar a <http://localhost:8083> se mostró el contenido del archivo local en vez de la página por defecto de nginx, confirmando que los cambios en mi máquina se reflejan directamente en el contenedor sin necesidad de reconstruir la imagen.

Imagen 12 — Comunicación entre contenedores por red personalizada.

```

PS C:\Users\roble> docker network create mired
2a1af0cdbfa8790c1cbc49c950d234a9a96815b9bc28f4ee3a264ab59a6ff2c6
PS C:\Users\roble> docker run -d --name servidor --network mired nginx
27e2b89f8313b7298410c8465530abe0681d1f5f429ea07accb8c05ab95bc150
PS C:\Users\roble> docker run --rm -it --network mired busybox ping -c 2
servidor
PING servidor (172.23.0.2): 56 data bytes
64 bytes from 172.23.0.2: seq=0 ttl=64 time=35.715 ms
64 bytes from 172.23.0.2: seq=1 ttl=64 time=0.149 ms

--- servidor ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.149/17.932/35.715 ms
PS C:\Users\roble>

```

-Creé una red propia (mired) y conecté un contenedor servidor (nginx) a ella. Luego lancé un contenedor temporal con busybox en la misma red y le hice ping al contenedor servidor usando su nombre en lugar de su IP. El resultado (2 paquetes

enviados, 2 recibidos, 0% de pérdida) confirma que Docker resuelve automáticamente los nombres de contenedores dentro de una red personalizada — algo que no ocurre en la red bridge por defecto.

Imagen 13 — Práctica final: red + volumen + persistencia completa.

```
PS C:\Users\roble> docker network create mired
2a1af0cdbfa8790c1cbc49c950d234a9a96815b9bc28f4ee3a264ab59a6ff2c6
PS C:\Users\roble> docker run -d --name servidor --network mired nginx
27e2b89f8313b7298410c8465530abe0681d1f5f429ea07accb8c05ab95bc150
PS C:\Users\roble> docker run --rm -it --network mired busybox ping -c 2
servidor
PING servidor (172.23.0.2): 56 data bytes
64 bytes from 172.23.0.2: seq=0 ttl=64 time=35.715 ms
64 bytes from 172.23.0.2: seq=1 ttl=64 time=0.149 ms

--- servidor ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.149/17.932/35.715 ms
PS C:\Users\roble> docker rm -f db
db
PS C:\Users\roble> docker network create red_demo
4a01b3ed016a0cf1f8070866096d6d56f4223cce5a07686a18364c7c113cfb43
PS C:\Users\roble> docker volume create vol_demo
vol_demo
PS C:\Users\roble> docker run -d --name db --network red_demo -v vol_demo:/var/lib/postgresql/data -e POSTGRES_PASSWORD=demo postgres:16
820b31c297c20d4b8b1fe6d928e6b3e7385947947aa106edb19e7355475f70f9
PS C:\Users\roble> docker exec -it db psql -U postgres
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: No such file or directory
Is the server running locally and accepting connections on that
socket?

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug db
    Learn more at https://docs.docker.com/go/debug-cli/
```

```
Windows PowerShell 5.1
tainer or image → docker debug db
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\roble> CREATE TABLE notas(t text);
t : El término 't' no se reconoce como nombre de un cmdlet, función,
archivo de script o programa ejecutable. Compruebe si escribió
correctamente el nombre o, si incluyó una ruta de acceso, compruebe
que dicha ruta es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 20
+ CREATE TABLE notas(t text);
+ ~
+ CategoryInfo          : ObjectNotFound: (t:String) [], CommandNo
tFoundException
+ FullyQualifiedErrorId : CommandNotFoundExeption

PS C:\Users\roble> INSERT INTO notas VALUES ('sobrevivi');
INSERT : El término 'INSERT' no se reconoce como nombre de un cmdlet,
función, archivo de script o programa ejecutable. Compruebe si
escribió correctamente el nombre o, si incluyó una ruta de acceso,
compruebe que dicha ruta es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ INSERT INTO notas VALUES ('sobrevivi');
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (INSERT:String) [], Comm
andNotFoundExeption
+ FullyQualifiedErrorId : CommandNotFoundExeption

PS C:\Users\roble> docker exec -it db psql -U postgres
psql (16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

postgres=# CREATE TABLE notas(t text);
INSERT INTO notas VALUES ('sobrevivi');
CREATE TABLE
INSERT 0 1
```

```

INSERT 0 1
postgres=# \q

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any con
tainer or image → docker debug db
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\roble> docker rm -f db
db
PS C:\Users\roble> docker run -d --name db --network red_demo -v vol_demo:/var/lib/postgresql/data -e POSTGRES_PASSWORD=demo postgres:16
782b94d6164766ef9f272e3b003385d233cfd01378570d356384115d046d9311
PS C:\Users\roble> docker exec -it db psql -U postgres -c "SELECT * FROM
notas;
t
-----
sobrevivi
(1 row)

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any con
tainer or image → docker debug db
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\roble>

```

-Realicé la práctica integradora del módulo: creé una red (red_demo) y un volumen (vol_demo), levanté un contenedor Postgres conectado a ambos, entré con docker exec y creé una tabla notas con el registro 'sobrevivi'. Luego eliminé el contenedor por completo y lo recreé exactamente igual, apuntando al mismo volumen. Al consultar la tabla nuevamente, el dato seguía presente, confirmando que la combinación de red + volumen + variables de entorno permite reconstruir el entorno del contenedor sin perder los datos.

Módulo 3 (Dockerfile)

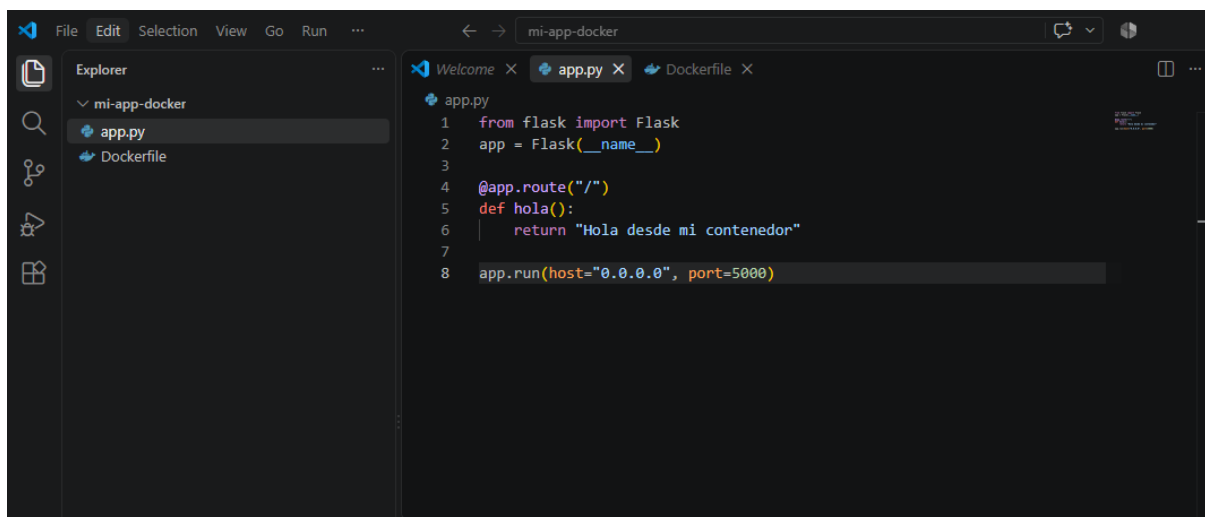
Imagen 14 — Creación de app.py y Dockerfile.

```
PS C:\Users\roble> mkdir mi-app-docker

Directorio: C:\Users\roble

Mode                LastWriteTime         Length Name
----                -
d-----          3/09/2026   3:10 p. m.         mi-app-docker

PS C:\Users\roble> cd mi-app-docker
PS C:\Users\roble\mi-app-docker> code .
PS C:\Users\roble\mi-app-docker>
```



-Creé la carpeta `mi-app-docker` con dos archivos: `app.py` (una app mínima de Flask que responde "Hola desde mi contenedor") y `Dockerfile` (la receta que le indica a Docker cómo construir la imagen: parte de `python:3.12-slim`, instala Flask, copia el código, expone el puerto 5000 y arranca la app).

Imagen 15 — Construcción y prueba de imagen propia con Dockerfile.

```
Windows PowerShell 5.1
PS C:\Users\roble\mi-app-docker> code .
PS C:\Users\roble\mi-app-docker> docker build -t mi-app:1.0 .
[+] Building 68.6s (10/10) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile      2.5s
=> => transferring dockerfile: 149B                      0.5s
=> [internal] load metadata for docker.io/library/python:3.12-slim 5.3s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                        0.2s
=> => transferring context: 2B                            0.1s
=> [1/4] FROM docker.io/library/python:3.12-slim@sha256:78387bc 32.5s
=> => resolve docker.io/library/python:3.12-slim@sha256:78387bc3 0.1s
=> => sha256:1560cfed01dc4c72c07f789d860078d7 12.12MB / 12.12MB 26.7s
=> => sha256:dbd0b7849e6c06b61e1fa6b7ffe053f246 4.27MB / 4.27MB 17.1s
=> => sha256:56235e4245636e401a28cf88944d543b29ee028 249B / 249B 0.4s
=> => extracting sha256:dbd0b7849e6c06b61e1fa6b7ffe053f246ce0075 6.3s
=> => extracting sha256:1560cfed01dc4c72c07f789d860078d74466a1f9 3.9s
=> => extracting sha256:56235e4245636e401a28cf88944d543b29ee028a 0.1s
=> [internal] load build context                        0.3s
=> => transferring context: 190B                          0.1s
=> [2/4] WORKDIR /app                                  0.4s
=> [3/4] RUN pip install flask                          17.7s
=> [4/4] COPY app.py .                                  0.7s
=> exporting to image                                   6.3s
=> => exporting layers                                     2.5s
=> => exporting manifest sha256:ead9961865e9376497e6dd668defab1b 0.1s
=> => exporting config sha256:24a7cab6c604aa575f8e8e4b04b2455740 0.1s
=> => exporting attestation manifest sha256:5e6cadd391d5e9783caa 0.1s
=> => exporting manifest list sha256:3588481b9c6832dda5f6b1dac30 0.1s
=> => naming to docker.io/library/mi-app:1.0              0.1s
=> => unpacking to docker.io/library/mi-app:1.0          2.7s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/iq1ll1sa9v8banf3u9cke4yg96
PS C:\Users\roble\mi-app-docker> docker run -d --name app1 -p 5001:5000
```

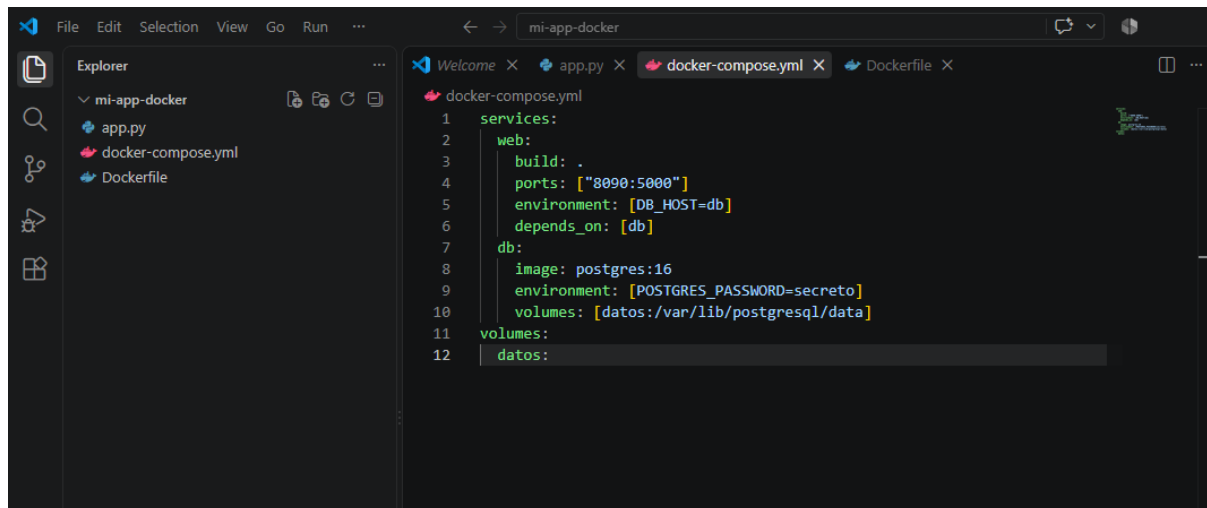
```
PS C:\Users\roble\mi-app-docker> docker run -d --name app1 -p 5001:5000
mi-app:1.0
17c5808402ce25325df0ca70226dc8368a6a94d1b82df79ec643d1753640088b
PS C:\Users\roble\mi-app-docker> docker ps --filter "name=app1"
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS         NAMES
17c5808402ce   mi-app:1.0     "python app.py"         21 seconds ago Up 16 se
conds         0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp   app1
```

Hola desde mi contenedor

-Construí mi propia imagen con `docker build -t mi-app:1.0 .`, usando el Dockerfile que preparé con Flask. Luego arranqué el contenedor con `docker run -d --name app1 -p 5001:5000 mi-app:1.0` (usando el puerto 5001 en mi máquina

para evitar conflicto con otro contenedor que ya usaba el 5000). Verifiqué con docker ps que quedó en estado "Up", y al entrar a `http://localhost:5001` la aplicación respondió correctamente con el mensaje "Hola desde mi contenedor", confirmando el ciclo completo: construir → ejecutar → probar.

Imagen 16 — Creación de docker-compose.yml.



-Creé el archivo `docker-compose.yml` en la misma carpeta del proyecto, definiendo dos servicios: `web` (que se construye con mi `Dockerfile` y se conecta a la base de datos vía `DB_HOST=db`) y `db` (usando la imagen `postgres:16` con un volumen para persistencia). El servicio `web` depende de `db` (`depends_on`), garantizando que la base de datos arranque primero.

Imagen 17 — Docker Compose: levantando web + base de datos con un solo comando.

```
PS C:\Users\roble\mi-app-docker> docker compose up -d
#1 [internal] load local bake definitions
#1 reading from stdin 515B 0.2s done
#1 DONE 0.2s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile:
#2 transferring dockerfile: 149B 0.1s done
#2 DONE 0.2s

#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 ...

#4 [auth] library/python:pull token for registry-1.docker.io
#4 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 3.8s

#5 [internal] load .dockerignore
#5 transferring context: 2B 0.0s done
#5 DONE 0.1s

#6 [internal] load build context
#6 transferring context: 28B 0.0s done
#6 DONE 0.1s

#7 [1/4] FROM docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
#7 resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
#7 resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea 0.1s done
#7 DONE 0.2s
```



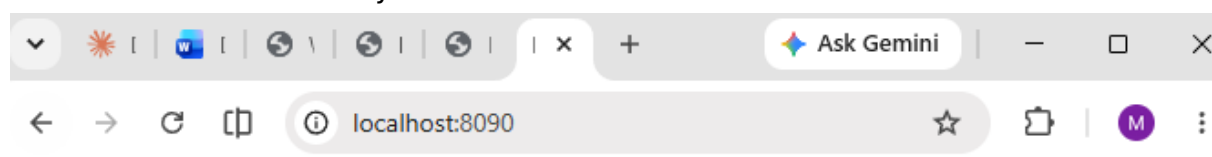
```

#11 exporting layers 0.0s done
#11 exporting manifest sha256:1c0cff9023e96d498f97abea356fbb2b0dd81fb7c
5dcdff1e79682a1eb1eda1f 0.1s done
#11 exporting config sha256:a33901f72588404afe749d77f069efa1aa7adcc1f52
93e0d704fc490df5bb6d8
#11 exporting config sha256:a33901f72588404afe749d77f069efa1aa7adcc1f52
93e0d704fc490df5bb6d8 0.0s done
#11 exporting attestation manifest sha256:e3624b5ab17cc2267dc0c553b483a
3772b8ca24e555b3e3d4bbf5ccd8f067620 0.1s done
#11 exporting manifest list sha256:574ee089ded19d2f9a36173b4a272edb3c78
8ac660d11dfc1ded1d2cf3b06411
#11 exporting manifest list sha256:574ee089ded19d2f9a36173b4a272edb3c78
8ac660d11dfc1ded1d2cf3b06411 0.0s done
#11 naming to docker.io/library/mi-app-docker-web:latest 0.0s done
#11 unpacking to docker.io/library/mi-app-docker-web:latest 0.1s done
#11 DONE 0.6s

#12 resolving provenance for metadata file
#12 DONE 0.1s
[+] up 5/5
✔Image mi-app-docker-web Built 11.5s
✔Network mi-app-docker_default Created 0.5s
✔Volume mi-app-docker_datos Created 0.2s
✔Container mi-app-docker-db-1 Started 6.1s
✔Container mi-app-docker-web-1 Started 9.4s
PS C:\Users\roble\mi-app-docker> docker compose ps
NAME                IMAGE              COMMAND              SERV
ICE    CREATED          STATUS            PORTS
mi-app-docker-db-1  postgres:16       "docker-entrypoint.s...  db
      21 seconds ago    Up 19 seconds    5432/tcp
mi-app-docker-web-1  mi-app-docker-web "python app.py"        web
      21 seconds ago    Up 14 seconds    0.0.0.0:8090->5000/tcp, [::]:809
0->5000/tcp

```

-Ejecuté `docker compose up -d` para levantar toda la aplicación definida en `docker-compose.yml`. Con un solo comando, Docker construyó la imagen de mi app, creó automáticamente una red y un

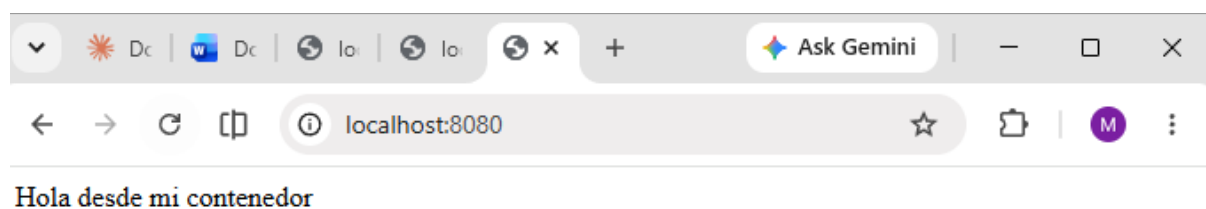


Hola desde mi contenedor

-volumen, y arrancó los dos contenedores (db con Postgres y web con mi app Flask) ya conectados entre sí. Verifiqué con `docker compose ps` que ambos servicios quedaron en estado "Up", y al entrar a `http://localhost:8090` la aplicación respondió correctamente. Esto demuestra la ventaja de Compose frente a levantar cada contenedor manualmente: todo el entorno (red, volumen, variables, dependencias) queda definido en un solo archivo reproducible.

Imagen 18 — Ejercicio integrador MiniBlog: persistencia tras down/up.

```
✓Volume mi-app-docker_datos Created 0.2s
✓Container mi-app-docker-db-1 Started 6.1s
✓Container mi-app-docker-web-1 Started 9.4s
PS C:\Users\roble\mi-app-docker> docker compose ps
NAME                IMAGE                COMMAND                SERV
ICE    CREATED          STATUS              PORTS
mi-app-docker-db-1  postgres:16         "docker-entrypoint.s..." db
                21 seconds ago     Up 19 seconds      5432/tcp
mi-app-docker-web-1 mi-app-docker-web   "python app.py"       web
                21 seconds ago     Up 14 seconds      0.0.0.0:8090->5000/tcp, [::]:809
0->5000/tcp
PS C:\Users\roble\mi-app-docker> docker compose down
[+] down 3/3
✓Container mi-app-docker-web-1 Removed 11.7s
✓Container mi-app-docker-db-1 Removed 1.3s
✓Network mi-app-docker_default Removed 0.4s
PS C:\Users\roble\mi-app-docker> docker compose up -d
[+] up 3/3
✓Network mi-app-docker_default Created 0.1s
✓Container mi-app-docker-db-1 Started 3.6s
✓Container mi-app-docker-web-1 Started 4.5s
PS C:\Users\roble\mi-app-docker> docker compose down
[+] down 3/3
✓Container mi-app-docker-web-1 Removed 11.9s
✓Container mi-app-docker-db-1 Removed 1.6s
✓Network mi-app-docker_default Removed 0.4s
PS C:\Users\roble\mi-app-docker> docker compose up -d
[+] up 3/3
✓Network mi-app-docker_default Created 0.1s
✓Container mi-app-docker-db-1 Started 1.9s
✓Container mi-app-docker-web-1 Started 3.7s
PS C:\Users\roble\mi-app-docker> curl.exe localhost:8080
Hola desde mi contenedor
PS C:\Users\roble\mi-app-docker>
```



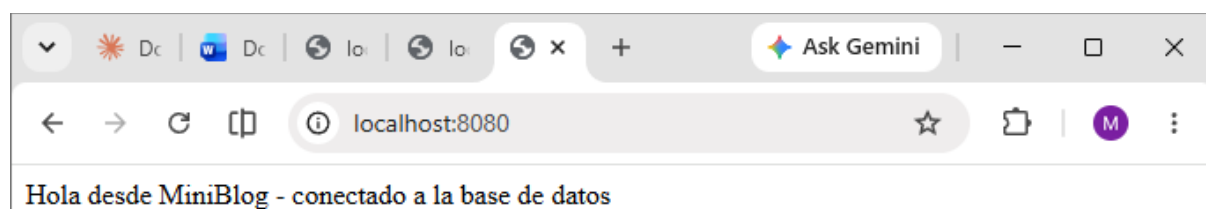
-Ajusté el puerto de mi docker-compose.yml a 8080 y levanté los servicios con `docker compose up -d`. Repetí el ciclo down → up dos veces para comprobar que la app se reconstruye correctamente cada vez. La prueba final con `curl.exe localhost:8080` confirmó que la aplicación sigue respondiendo "Hola desde mi contenedor" tras destruir y recrear los contenedores, cumpliendo los criterios del ejercicio integrador "MiniBlog en contenedores".

Imagen 19 — MiniBlog con conexión real a la base de datos.

```
PS C:\Users\roble\mi-app-docker> docker compose up -d
[+] up 3/3
✓Network mi-app-docker_default Created 0.1s
✓Container mi-app-docker-db-1 Started 3.6s
✓Container mi-app-docker-web-1 Started 4.5s
PS C:\Users\roble\mi-app-docker> docker compose down
[+] down 3/3
✓Container mi-app-docker-web-1 Removed 11.9s
✓Container mi-app-docker-db-1 Removed 1.6s
✓Network mi-app-docker_default Removed 0.4s
PS C:\Users\roble\mi-app-docker> docker compose up -d
[+] up 3/3
✓Network mi-app-docker_default Created 0.1s
✓Container mi-app-docker-db-1 Started 1.9s
✓Container mi-app-docker-web-1 Started 3.7s
PS C:\Users\roble\mi-app-docker> curl.exe localhost:8080
Hola desde mi contenedor
PS C:\Users\roble\mi-app-docker> docker compose down
[+] down 3/3
✓Container mi-app-docker-web-1 Removed 11.9s
✓Container mi-app-docker-db-1 Removed 1.8s
✓Network mi-app-docker_default Removed 0.4s
PS C:\Users\roble\mi-app-docker> docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 515B 0.1s done
#1 DONE 0.1s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile:
#2 transferring dockerfile: 167B 0.3s done
#2 DONE 0.3s

#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 ...
```

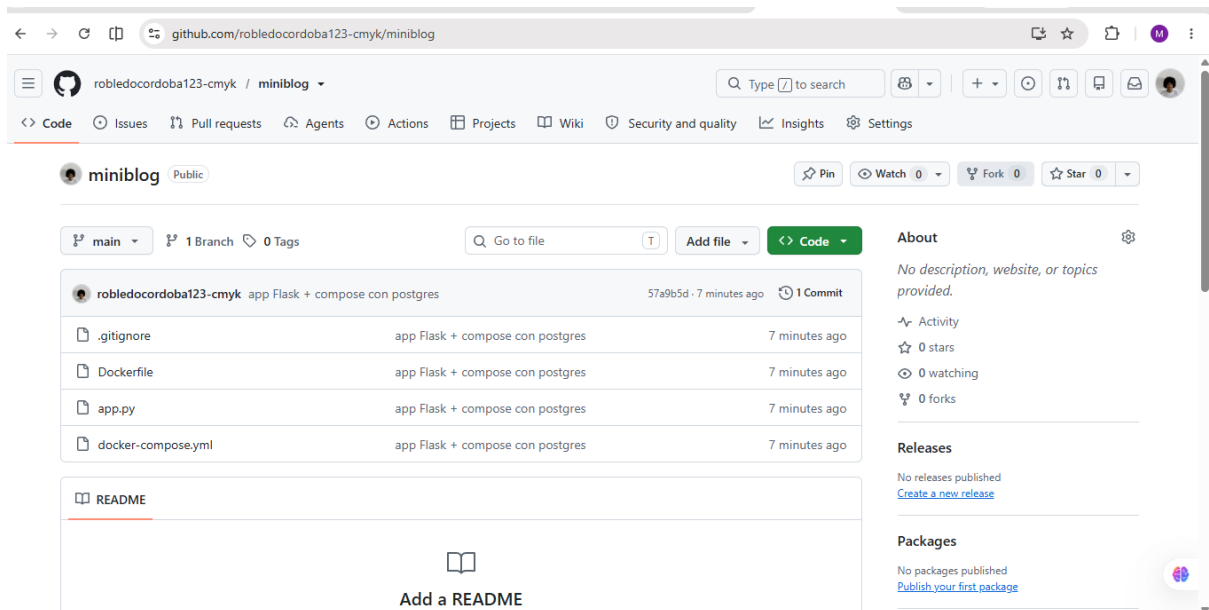


-Actualicé el Dockerfile agregando psycopg2-binary (librería para conectar Python con PostgreSQL) y modifiqué app.py para que la ruta principal abra una conexión real a la base de datos usando DB_HOST=db (resuelto automáticamente por la red que crea Docker Compose). Reconstruí la imagen con `docker compose up -d --build` y verifiqué en el navegador que la app responde "Hola desde MiniBlog - conectado a la base de datos", confirmando que la aplicación web y la base de datos PostgreSQL están correctamente integradas dentro del mismo entorno de contenedores.

Imagen 20 — Repositorio Git y subida a GitHub.

```
PS C:\Users\roble\mi-app-docker> git init
Initialized empty Git repository in C:/Users/roble/mi-app-docker/.git/
PS C:\Users\roble\mi-app-docker> git add .
PS C:\Users\roble\mi-app-docker> git commit -m "app Flask + compose con
postgres"
[master (root-commit) 57a9b5d] app Flask + compose con postgres
4 files changed, 42 insertions(+)
create mode 100644 .gitignore
create mode 100644 Dockerfile
create mode 100644 app.py
create mode 100644 docker-compose.yml
PS C:\Users\roble\mi-app-docker> git log --oneline
57a9b5d (HEAD -> master) app Flask + compose con postgres
PS C:\Users\roble\mi-app-docker> git remote add origin https://github.c
om/robledocordoba123-cmyk/miniblog.git
PS C:\Users\roble\mi-app-docker> git branch -M main
PS C:\Users\roble\mi-app-docker> git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 964 bytes | 241.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/robledocordoba123-cmyk/miniblog.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\roble\mi-app-docker>
```

-Iniciale un repositorio Git local con `git init` dentro de la carpeta del proyecto, agregué un archivo `.gitignore` para excluir archivos sensibles y temporales, y confirmé los archivos del proyecto (`Dockerfile`, `app.py`, `docker-compose.yml`) con `git commit`. Luego conecté el repositorio local con uno vacío creado en GitHub (`git remote add origin`), renombré la rama a `main` y subí el código con `git push -u origin main`. El proyecto quedó bajo control de versiones y disponible en la nube, listo para que cualquier mejora futura quede registrada en el historial.



-Verifiqué en GitHub que el repositorio miniblog quedó creado con los 4 archivos del proyecto correctamente subidos, confirmando que el control de versiones está funcionando de extremo a extremo (repo local → GitHub).

Imagen 21 — Ramas, commits y fusión (merge) con main

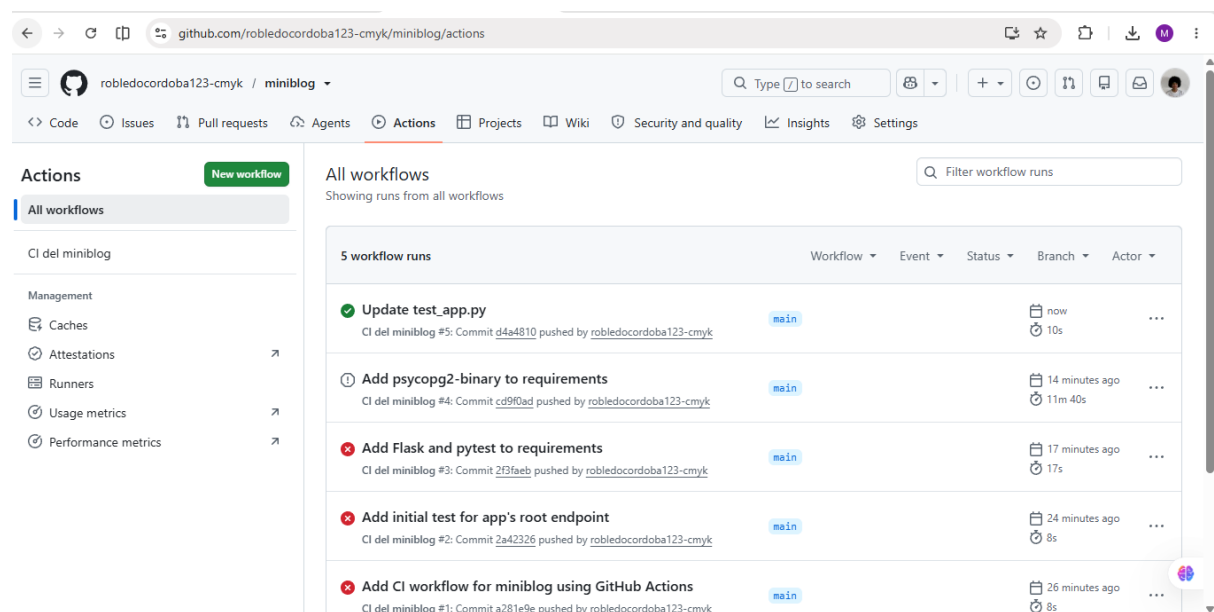
```
PS C:\Users\roble\mi-app-docker> git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\roble\mi-app-docker> git merge feature/login
Updating 57a9b5d..0202f6e
Fast-forward
 app.py | 4 +++-
 1 file changed, 3 insertions(+), 1 deletion(-)
PS C:\Users\roble\mi-app-docker> git log --oneline
0202f6e (HEAD -> main, feature/login) agrega login
57a9b5d (origin/main) app Flask + compose con postgres
PS C:\Users\roble\mi-app-docker> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 320 bytes | 35.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/robledocordoba123-cmyk/miniblog.git
 57a9b5d..0202f6e  main -> main
PS C:\Users\roble\mi-app-docker> |
```

-Creé una rama nueva llamada feature/login con `git switch -c feature/login` para trabajar de forma aislada sin afectar la rama principal. Hice un cambio en `app.py`, lo confirmé con `git commit -am "agrega login"`, y luego regresé a main con `git switch main` para fusionar los cambios usando `git merge feature/login`. Git

realizó un *fast-forward*, es decir, movió el puntero de `main` directamente hasta el commit de la rama sin necesidad de crear un commit de fusión adicional, porque `main` no tenía cambios nuevos mientras trabajaba en la rama. Confirmé el resultado con `git log --oneline`, donde se ve que `main` y `feature/login` apuntan al mismo commit (`0202f6e` agrega `login`).

-Subí el merge a GitHub con `git push origin main`, sincronizando la rama `main` local con el repositorio remoto para que refleje el último commit.

Imagen 22 — Ejecución del pipeline de integración continua (CI) en GitHub Actions.



-Vista general de la pestaña **Actions** en GitHub, mostrando el historial de ejecuciones del repositorio tras solucionar las dependencias en el archivo `requirements.txt` y ajustar las pruebas unitarias. Se observa el último commit registrado (`Update test_app.py`) marcado con un indicador verde de éxito (✓), lo que confirma que el pipeline de integración continua se ejecutó, instaló las librerías necesarias y validó las pruebas de forma automatizada en el entorno virtual de Ubuntu sin errores.